

U.S. SEMICONDUCTOR: TRADE & LABOR ANALYSIS

U.S. Semiconductor Imports — Source Data & Cleaning Process | 2025

Overview

This document describes the data cleaning steps applied to the U.S. semiconductor imports dataset (TradeData_1_21_2026_21_51_10.csv) sourced from UN Comtrade. The objective was to normalize quantities, handle missing values consistently, convert critical fields to numeric types, and produce a file that is transparent, auditable, and ready for trade-partner and budgetary analysis. The cleaned output is saved as Cleaned Imports USA.csv.

Step 1 — Reading the Raw Dataset

Objective

Load the original imports dataset into a pandas DataFrame for inspection and transformation.

Code

```
df = pd.read_csv("TradeData_1_21_2026_21_51_10.csv")
```

Key Design Choice

- **Source file:** The raw file contained multiple fields, some of which were redundant, inconsistent, or filled with placeholder values such as N/A. Loading into pandas provided full visibility before any transformation.

Step 2 — Converting Key Columns to Numeric

Objective

Ensure that quantity, weight, and value fields are correctly typed before any calculations or aggregations are performed.

Code

```
for col in ["qty", "altQty", "netWgt", "grossWgt", "cifvalue", "fobvalue", "primaryValue"]:  
    df[col] = pd.to_numeric(df[col], errors="coerce")
```

Key Design Choice

- **errors='coerce':** Safely converts invalid entries to NaN rather than raising an exception, allowing the analysis to continue while flagging problematic values for review.

Step 3 — Normalizing Quantities

Objective

Create a unified qty_standard column that makes quantity values comparable across records that use different reporting units (u, 1000u, kg).

Code

```
def normalize_qty(row):  
    unit = row["altQtyUnitAbbr"] if row["altQtyUnitAbbr"] != "N/A" else  
row["qtyUnitAbbr"]  
    qty = row["qty"]  
    if unit == "u":  
        return qty
```

```

elif unit == "1000u": return qty * 1000
elif unit == "kg":    return qty
else:                return np.nan

df["qty_standard"] = df.apply(normalize_qty, axis=1)
df["qty_kg"] = df["netWgt"].fillna(0) # direct reference to net weight

```

Key Design Choice

- **qty_standard:** Unifies different reporting units into a single comparable column. qty_kg is added as a direct reference to netWgt, with missing values replaced by zero for consistency.

Step 4 — Converting Booleans and Labeling Data Quality

Objective

Standardize boolean flag fields and create a data_quality column to clearly communicate whether weight data was estimated or reported.

Code

```

df["isNetWgtEstimated"] = (
    df["isNetWgtEstimated"].astype(str).str.upper()
    .map({"TRUE": True, "FALSE": False})
)
df["data_quality"] = df["isNetWgtEstimated"].map({True: "Estimated", False:
"Reported"})

```

Key Design Choice

- **data_quality column:** Makes the reliability of weight data immediately visible in the dataset without requiring users to interpret raw boolean flags.

Step 5 — Selective Row Cleaning

Objective

Remove rows where all critical fields are simultaneously missing, while preserving records with partial information.

Code

```

critical_cols = ["reporterISO", "motDesc", "qty", "cifvalue"]
df = df.dropna(subset=critical_cols, how="all")

# Fill numeric NaN with 0; fill text NaN with "Unknown"
df[numeric_cols] = df[numeric_cols].fillna(0)
df[text_cols] = df[text_cols].fillna("Unknown")

```

Key Design Choice

- **how='all':** Drops only rows where every critical field is missing simultaneously. Records with partial information are preserved for transparency and downstream analysis.
- **'Unknown' fill:** Replacing N/A with 'Unknown' in text fields improves human readability and avoids ambiguity in grouping and filtering operations.

Step 6 — Removing Duplicates

Objective

Eliminate duplicate rows to ensure the dataset is clean and each record is unique.

Code

```
df = df.drop_duplicates()
```

Step 7 — Dropping Redundant Columns

Objective

Remove columns identified during EDA as redundant or irrelevant to semiconductor trade analysis, reducing noise and streamlining the dataset.

Code

```
cols_to_drop = ['primaryValue', 'partner2Code', 'partner2ISO',  
               'partner2Desc', 'grossWgt', 'mosCode']  
df = df.drop(columns=cols_to_drop)
```

Rationale for Each Dropped Column

Column	Reason for Removal
primaryValue	Duplicates fobvalue — no additional analytical value
partner2Code	Not relevant for bilateral U.S. import analysis
partner2ISO	Not relevant for bilateral U.S. import analysis
partner2Desc	Not relevant for bilateral U.S. import analysis
grossWgt	Consistently empty or zero in this dataset
mosCode	Metadata field with no analytical use

Step 8 — Saving the Cleaned Dataset

Objective

Export the cleaned dataset as a structured CSV file ready for EDA, SQL loading, and visualization.

Code

```
df.to_csv("Cleaned Imports USA.csv", index=False)
```

Output Summary

Metric	Value
Output file	Cleaned Imports USA.csv
Rows retained	373
Key fields	reporter, partner, transport mode, normalized quantity, CIF value, FOB value
Data quality column	Estimated / Reported (derived from isNetWgtEstimated)

Key Decisions and Rationale

- **Weights handled carefully:** netWgt is retained but missing values replaced with zero. Data quality is documented via the new data_quality column.
- **Quantity normalization:** qty_standard unifies different reporting units, ensuring comparability across all records.
- **Text clarity:** N/A values replaced with 'Unknown' to make the dataset more human-readable and safe for grouping operations.
- **Selective cleaning:** Rows missing all critical fields are dropped; records with partial data are preserved for transparency.
- **Redundancy reduced:** Duplicate rows and irrelevant columns removed to streamline the dataset without losing meaningful information.

Data Source

[UN Comtrade](#) — TradeData_1_21_2026_21_51_10.csv